

Teoria da Computação

Autômatos Finitos e Linguagens Regulares

Prof. Renê R. Veloso

Expressões Regulares

As expressões regulares têm um papel importante em aplicações da ciência da computação. Em aplicações envolvendo texto, usuários podem querer fazer busca por cadeias que satisfazem certos padrões. Expressões regulares provêem um método poderoso para descrever tais padrões. Utilitários tais como AWK e GREP no UNIX, linguagens de programação modernas tais como PERL, e editores de texto, todos eles provêem mecanismos para a descrição de padrões usando expressões regulares.

Exemplo: regex em linguagens de programação

Expressões Regulares

DEFINIÇÃO 1.52

Digamos que R é uma *expressão regular* se R for

1. a para algum a no alfabeto Σ ,
2. ϵ ,
3. $\emptyset$,
4. $(R_1 \cup R_2)$, onde R_1 e R_2 são expressões regulares,
5. $(R_1 \circ R_2)$, onde R_1 e R_2 são expressões regulares, ou
6. (R_1^*) , onde R_1 é uma expressão regular.

Ordem de
precedência



Nos itens 1 e 2, as expressões regulares a e ϵ representam as linguagens $\{a\}$ e $\{\epsilon\}$, respectivamente. No item 3, a expressão regular $\emptyset$ representa a linguagem vazia. Nos itens 4, 5 e 6, as expressões representam as linguagens obtidas tomando-se a união ou concatenação das linguagens R_1 e R_2 , ou a estrela da linguagem R_1 , respectivamente.

Expressões Regulares

EXEMPLO 1.51

Um outro exemplo de uma expressão regular é

$$(0 \cup 1)^*$$

Ela começa com a linguagem $(0 \cup 1)$ e aplica a operação $*$. O valor dessa expressão é a linguagem consistindo de todas as possíveis cadeias de 0s e 1s. Se $\Sigma = \{0,1\}$, podemos escrever Σ como abreviação para a expressão regular $(0 \cup 1)$. Mais genericamente, se Σ for um alfabeto qualquer, a expressão regular Σ descreve a linguagem consistindo de todas as cadeias de comprimento 1 sobre esse alfabeto, e Σ^* descreve a linguagem consistindo de todas as cadeias sobre esse alfabeto. Similarmente Σ^*1 é a linguagem que contém todas as cadeias que terminam em 1. A linguagem $(0\Sigma^*) \cup (\Sigma^*1)$ consiste de todas as cadeias que começam com 0 ou terminam com 1. ■

Expressões Regulares

Exemplo:

$$L = \{w \in \{0,1\}^* : w = 0^K \text{ ou } w = 10^K, \text{ com } K \geq 0\}$$

$$\Rightarrow \{0\}^* \cup \{1\}\{0\}^*$$

$$\Rightarrow 0^* \cup 10^*$$

$$\Rightarrow (\epsilon \cup 1)0^*$$

Linguagem das cadeias que podem ou não começar com 1, seguido por zero ou mais 0's.

Expressões Regulares

EXEMPLO 1.53

Nas instâncias abaixo assumimos que o alfabeto Σ é $\{0,1\}$.

1. $0^*10^* = \{w \mid w \text{ contém um único } 1\}$.
2. $\Sigma^*1\Sigma^* = \{w \mid w \text{ tem pelo menos um símbolo } 1\}$.
3. $\Sigma^*001\Sigma^* = \{w \mid w \text{ contém a cadeia } 001 \text{ como uma subcadeia}\}$.
4. $1^*(01^+)^* = \{w \mid \text{todo } 0 \text{ em } w \text{ é seguido por pelo menos um } 1\}$.
5. $(\Sigma\Sigma)^* = \{w \mid w \text{ é uma cadeia de comprimento par}\}.$ ⁵
6. $(\Sigma\Sigma\Sigma)^* = \{w \mid \text{o comprimento de } w \text{ é um múltiplo de três}\}$.
7. $01 \cup 10 = \{01, 10\}$.

Expressões Regulares

EXEMPLO 1.53

Nas instâncias abaixo assumimos que o alfabeto Σ é $\{0,1\}$.

8. $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1 = \{w \mid w \text{ começa e termina com o mesmo símbolo}\}.$

9. $(0 \cup \epsilon)1^* = 01^* \cup 1^*.$

A expressão $0 \cup \epsilon$ descreve a linguagem $\{0, \epsilon\}$, portanto a operação de concatenação adiciona 0 ou ϵ antes de toda cadeia em 1^* .

10. $(0 \cup \epsilon)(1 \cup \epsilon) = \{\epsilon, 0, 1, 01\}.$

11. $1^*\emptyset = \emptyset.$

Concatenar o conjunto vazio a qualquer conjunto produz o conjunto vazio.

12. $\emptyset^* = \{\epsilon\}.$

A operação estrela junta qualquer número de cadeias da linguagem para obter uma cadeia no resultado. Se a linguagem for vazia, a operação estrela pode juntar 0 cadeias, dando apenas a cadeia vazia.

Expressões Regulares: equivalências

1. $r \cup s = s \cup r$

2. $r \cup \emptyset = r$

3. $r \cup r = r$

4. $r\varepsilon = \varepsilon r = r$

5. $r\emptyset = \emptyset r = \emptyset$

6. $(r \cup s) t = rt \cup st$

7. $r (s \cup t) = rs \cup rt$

8. $r^{**} = r^*$

9. $\emptyset^* = \varepsilon$

10. $\varepsilon^* = \varepsilon$

11. $r^* r^* = r^*$

12. $rr^* = r^*r$

13. $(r^* \cup s)^* = (r \cup s)^*$

14. $(r^* s^*)^* = (r \cup s)^*$

15. $r^* (r \cup s)^* = (r \cup s)^*$

16. $(r \cup s)^* r^* = (r \cup s)^*$

Expressões Regulares: meta caracteres

- ?
 - Zero ou uma ocorrência
 - $ab?$ equivale a $ab \cup \varepsilon$
 - +
 - Equivale a uma ou mais ocorrências
 - ab^+ equivale a abb^*
-
- [...]
 - Equivale a um intervalo ou lista
 - $a[0 - 9]$ equivale a $a(0 \cup 1 \cup 2 \cup 3 \cup 4 \cup 5 \cup 6 \cup 7 \cup 8 \cup 9)$
 - $[a - zA - Z]$ equivale a uma letra em maiúsculo ou minúsculo

Exercícios de fixação

Apresente ERs para as linguagens a seguir:

- a) Conjunto das palavras que começam com 0
- b) Conjunto das palavras com, no mínimo, um 0
- c) Conjunto das palavras que contêm 00 ou 11
- d) Conjunto das palavras que terminam com 0 seguido de um número ímpar de 1's consecutivos
- e) Conjunto das palavras de tamanho ímpar
- f) Conjunto das palavras que começam com 0, seguidos de zero ou mais 1's e terminadas com 2

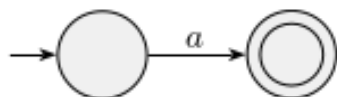
Equivalência com AFs: de ER para AFN

TEOREMA 1.54

Uma linguagem é regular se e somente se alguma expressão regular a descreve.

Para converter de uma expressão regular para um AFN:

1. $R = a$ para algum a em Σ . Então $L(R) = \{a\}$, e o seguinte AFN reconhece $L(R)$.



2. $R = \epsilon$. Então $L(R) = \{\epsilon\}$, e o seguinte AFN reconhece $L(R)$.



3. $R = \emptyset$. Então $L(R) = \emptyset$, e o seguinte AFN reconhece $L(R)$.



4. $R = R_1 \cup R_2$.

5. $R = R_1 \circ R_2$.

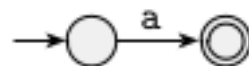
6. $R = R_1^*$.

de ER para AFN

EXEMPLO 1.56

Convertemos a expressão regular $(ab \cup a)^*$

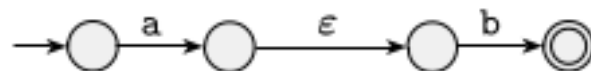
a



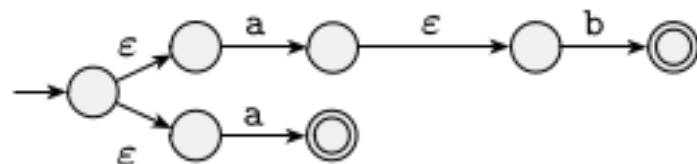
b



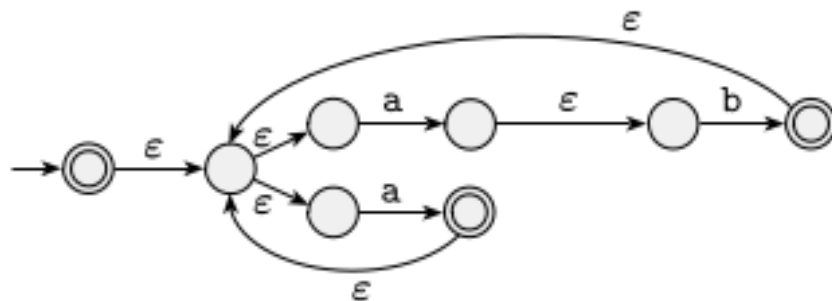
ab



$ab \cup a$



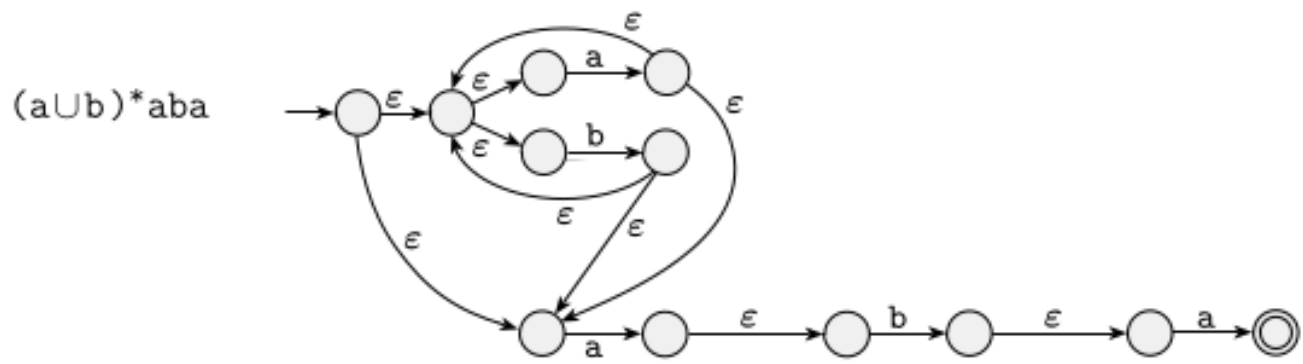
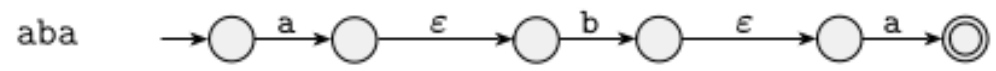
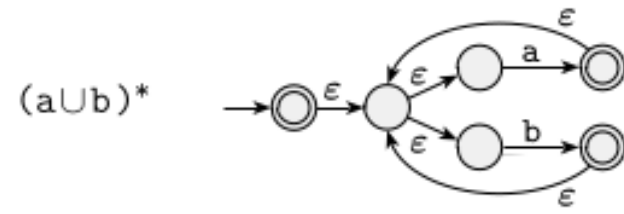
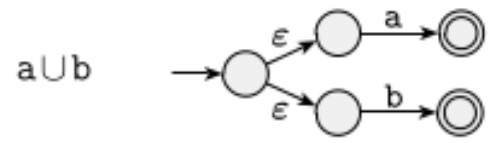
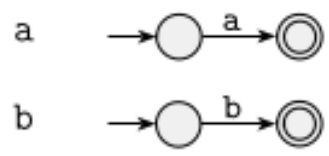
$(ab \cup a)^*$



de ER para AFN

EXEMPLO 1.58

$(a \cup b)^* aba$



Equivalência com AFs: de AFN para ER

TEOREMA 1.54

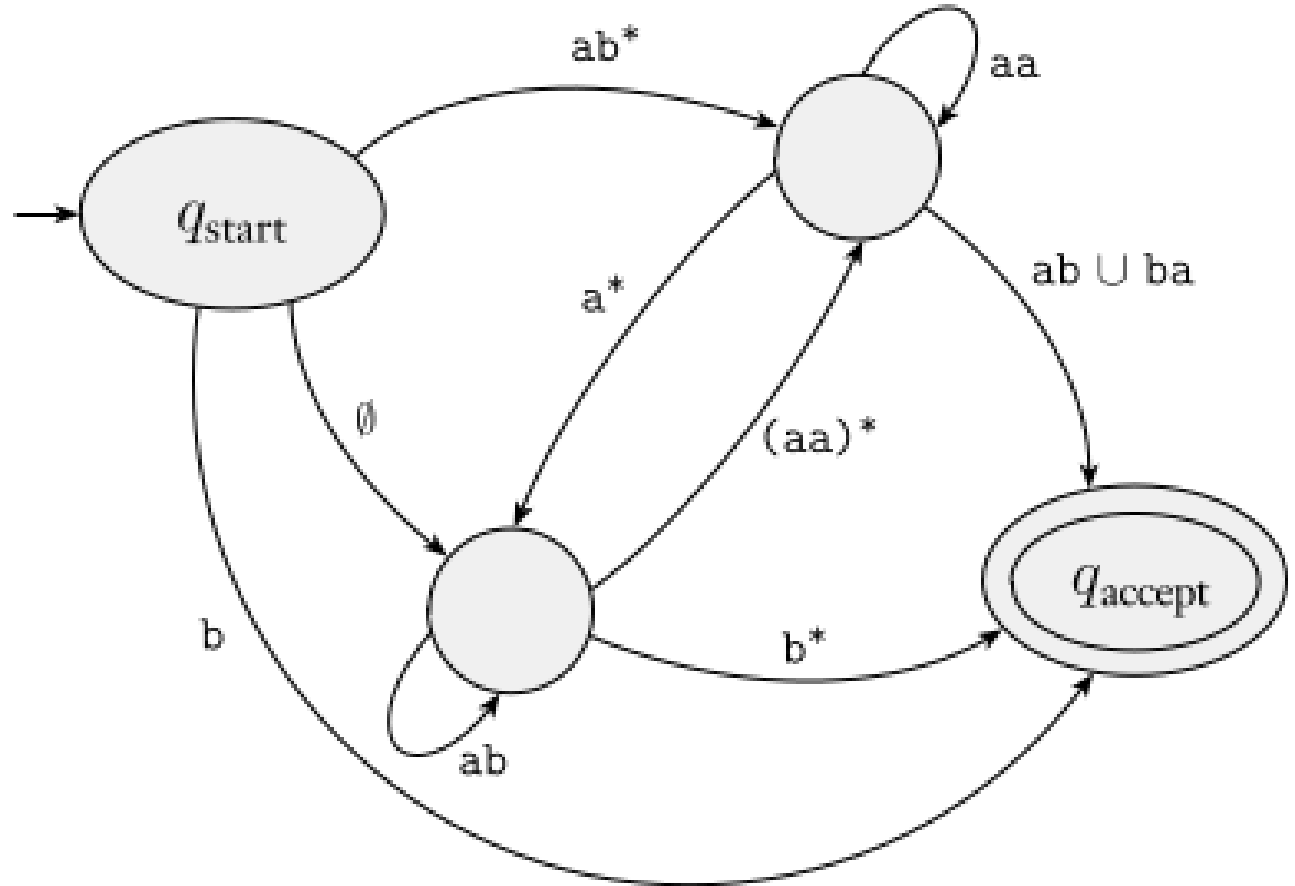
Uma linguagem é regular se e somente se alguma expressão regular a descreve.

Para converter um AFN para uma expressão regular precisamos de uma formalização denominado Autômato Finito Não-determinístico Generalizado (AFNG):

- AFN cujas transições são rotuladas por expressões regulares, ao invés de símbolos do alfabeto
- AFNG lê blocos de símbolos da fita, não necessariamente um de cada vez como ocorre com os AFNs;
- Se uma transição de um estado p_i para o estado p_j é rotulada por uma expressão regular R , então essa transição ocorre lendo-se um bloco de símbolos da entrada que constituam uma cadeia descrita por R .
- Um AFNG aceita uma entrada se sua computação puder resultar em um estado de aceitação ao final da leitura.

Equivalência com AFs: de AFN para ER

Exemplo de AFNG:



Equivalência com AFs: de AFN para ER

Autômato Finito Não-determinístico Generalizado (AFNG)

- Condições:
 - Estado inicial possui setas de transições saindo para todos os demais estados, mas não possui setas de transições chegando de nenhum estado
 - Há apenas um único estado de aceitação, que possui setas chegando de todos os demais estados mas não setas saindo para outros estados
 - Exceto para os estados inicial e de aceitação, existe sempre uma seta saindo de todo estado para os demais estados, e também uma seta de cada estado para si mesmo.

Equivalência com AFs: de AFN para ER

Definição 1.33

Um *autômato finito não-determinístico generalizado*, $(Q, \Sigma, \delta, q_{\text{início}}, q_{\text{aceita}})$, é uma 5-upla onde

1. Q é o conjunto finito de estados,
2. Σ é o alfabeto de entrada,
3. $\delta : (Q - \{q_{\text{aceita}}\}) \times (Q - \{q_{\text{início}}\}) \longrightarrow \mathcal{R}$ é a função de transição,
4. $q_{\text{início}}$ é o estado inicial, e
5. q_{aceita} é o estado de aceitação.

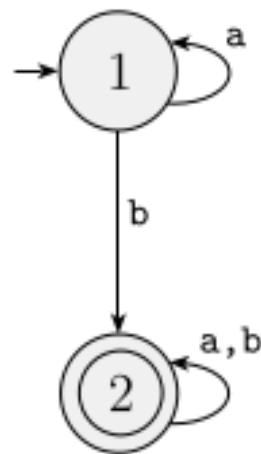
Um AFNG aceita uma cadeia w em Σ^* se $w = w_1 w_2 \cdots w_k$, onde cada w_i está em Σ^* e uma seqüência de estados $q_0, q_1, \dots, q_k$ existe tal que

1. $q_0 = q_{\text{início}}$ é o estado inicial,
2. $q_k = q_{\text{aceita}}$ é o estado de aceitação, e
3. para cada i , temos $w_i \in L(R_i)$, onde $R_i = \delta(q_{i-1}, q_i)$; em outras palavras, R_i é a expressão sobre a seta de q_{i-1} para q_i .

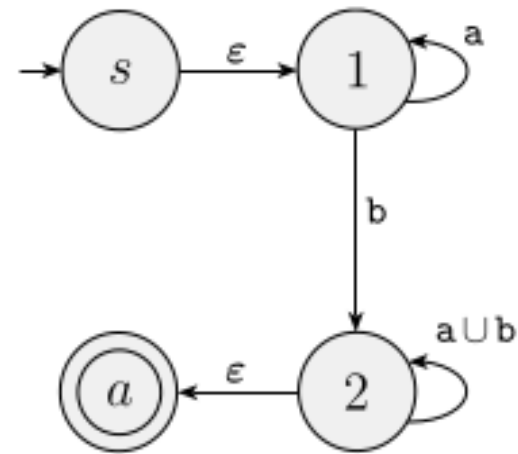
De AFN para AFNG:

... adicionamos um novo estado inicial com uma seta ϵ ao antigo estado inicial e um novo estado de aceitação com setas ϵ a partir dos antigos estados de aceitação. Se quaisquer setas têm múltiplos rótulos (ou se existem múltiplas setas indo entre os mesmos dois estados na mesma direção), substituímos cada uma por uma única seta cujo rótulo é a união dos rótulos anteriores.

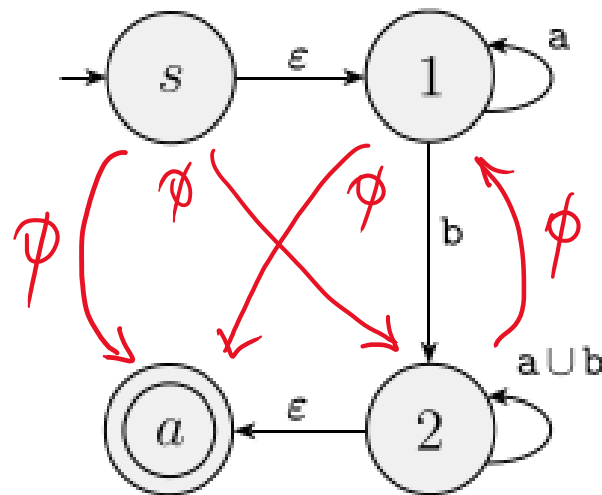
Finalmente, adicionamos setas rotuladas com $\emptyset$ entre TODOS estados que não tinham setas, exceto chegando no estado inicial e saindo do estado final.



(a)

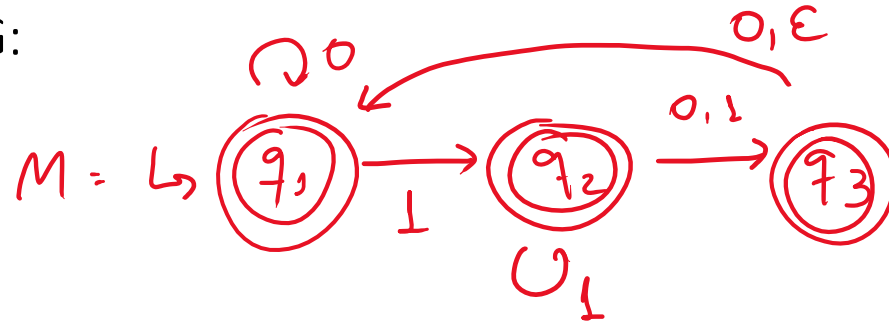


(b)



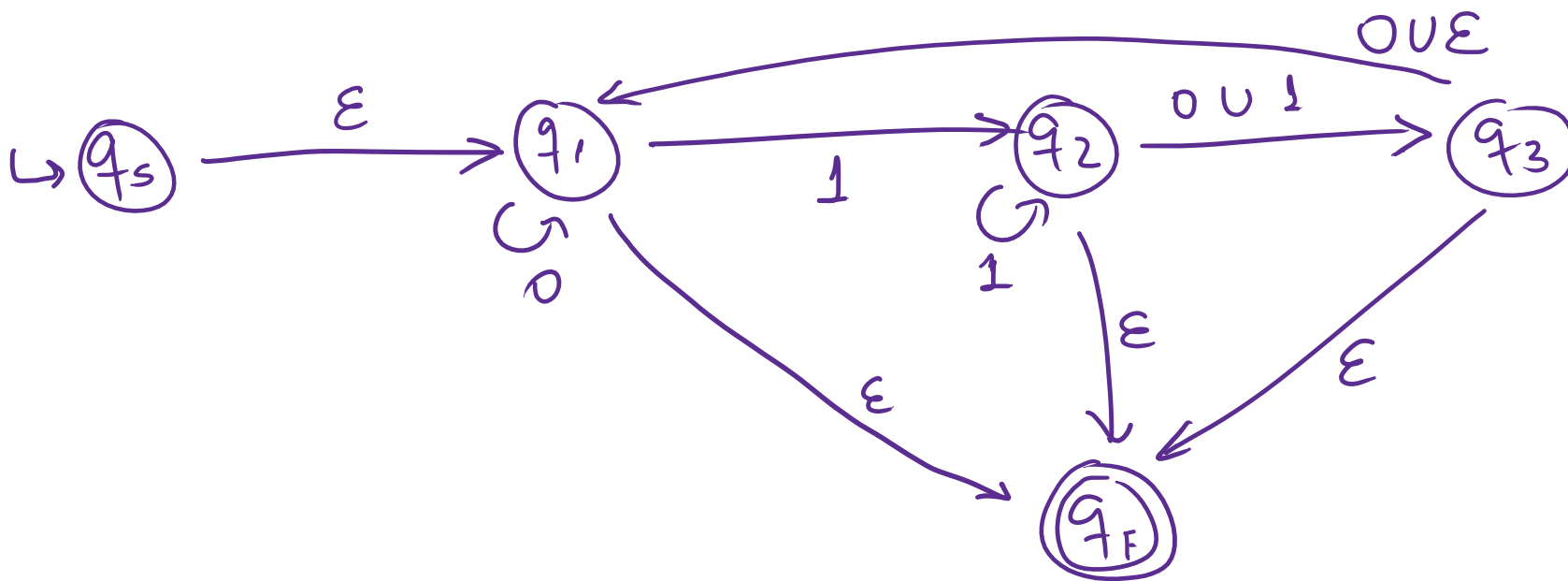
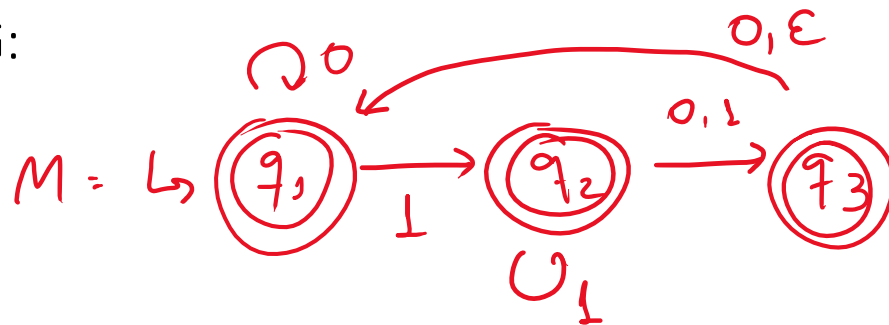
De AFN para AFNG:

Outro exemplo



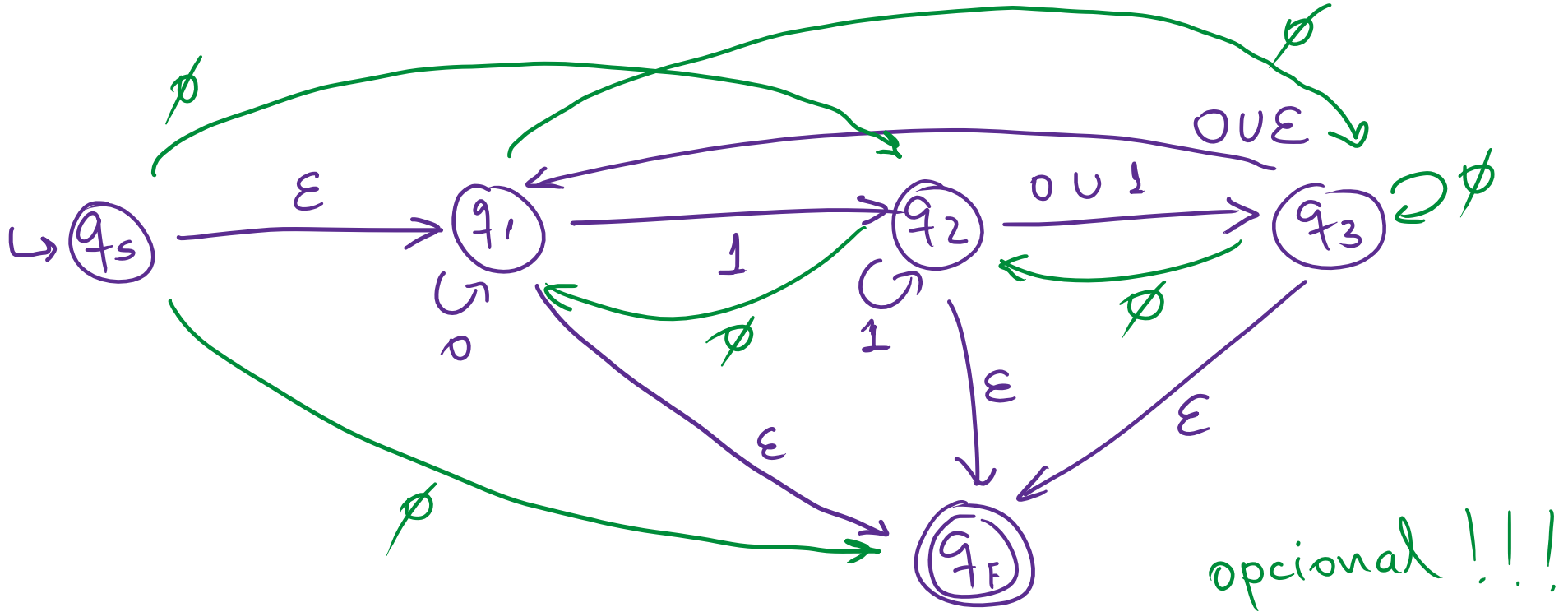
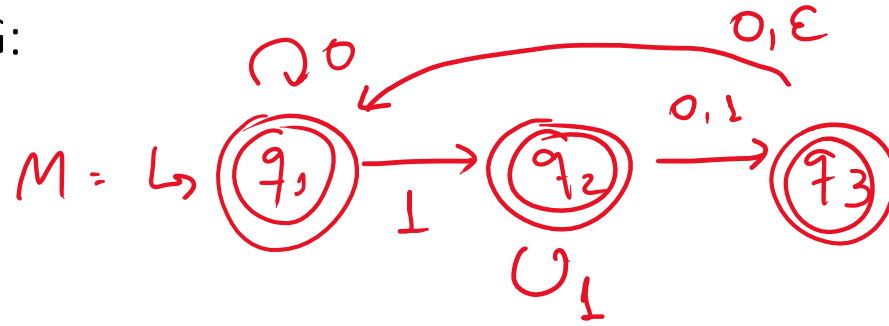
De AFN para AFNG:

Outro exemplo



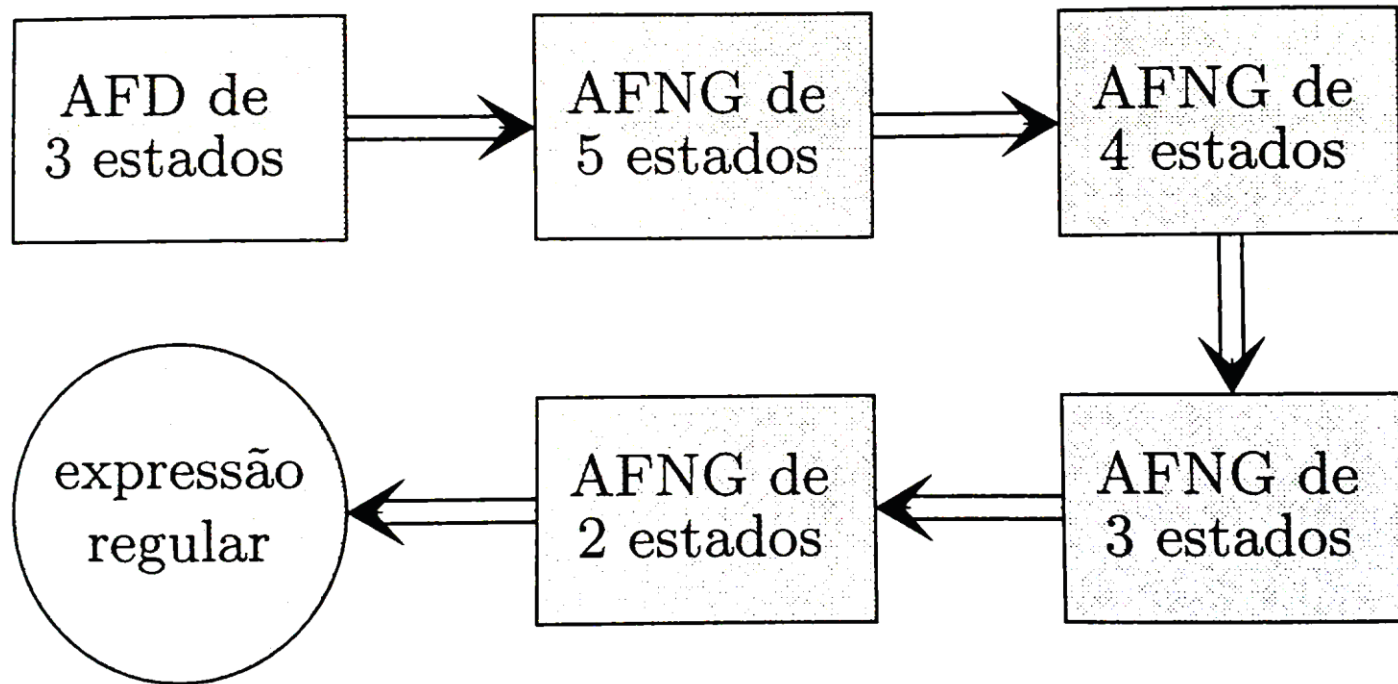
De AFN para AFNG:

Outro exemplo



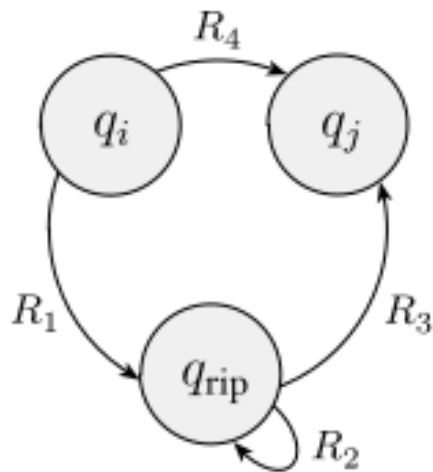
Equivalência com AFs: de AFN para ER

Etapas típicas da conversão para o caso de um AFD de 3 estados:

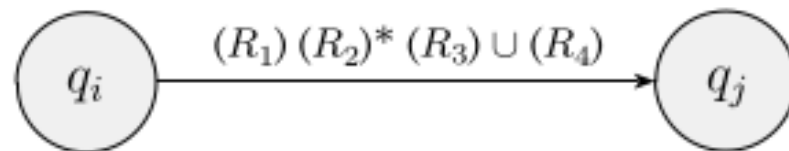


Equivalência com AFs: de AFN para ER

Construção de um AFNG
equivalente com um estado a
menos:



before



after

CONVERT(G):

Algoritmo de conversão

1. Let k be the number of states of G .
2. If $k = 2$, then G must consist of a start state, an accept state, and a single arrow connecting them and labeled with a regular expression R .
Return the expression R .
3. If $k > 2$, we select any state $q_{\text{rip}} \in Q$ different from q_{inicio} and q_{aceita} and let G' be the AFNG $(Q', \Sigma, \delta', q_{\text{inicio}}, q_{\text{aceita}})$, where

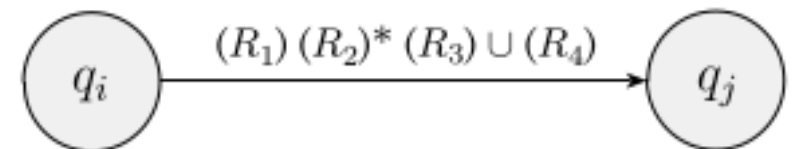
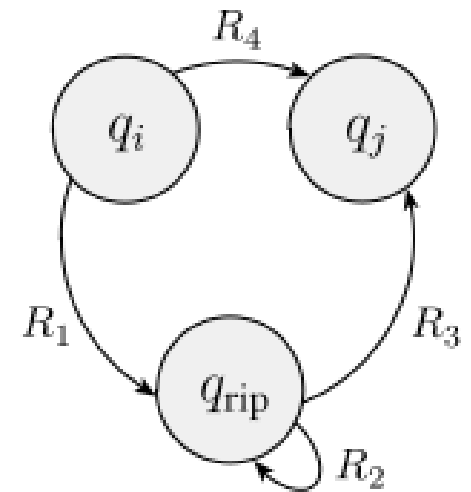
$$Q' = Q - \{q_{\text{rip}}\},$$

and for any $q_i \in Q' - \{q_{\text{aceita}}\}$ and any $q_j \in Q' - \{q_{\text{inicio}}\}$ let

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4),$$

for $R_1 = \delta(q_i, q_{\text{rip}})$, $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$, $R_3 = \delta(q_{\text{rip}}, q_j)$, and $R_4 = \delta(q_i, q_j)$.

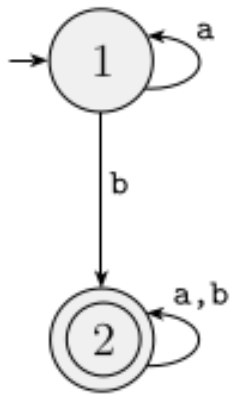
4. Compute CONVERT(G') and return this value.



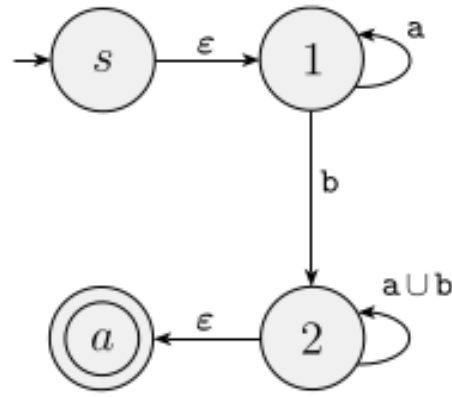
Algoritmo de conversão

Para remover o estado 2.

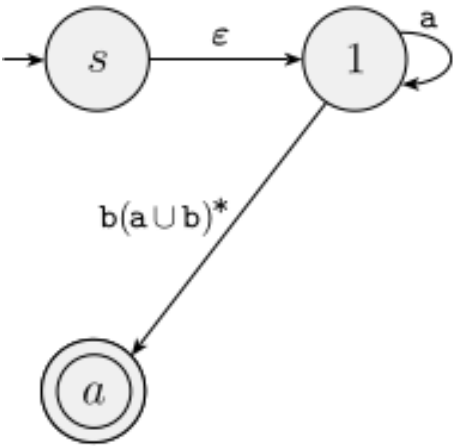
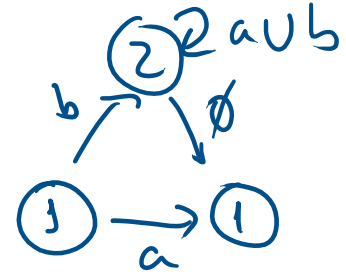
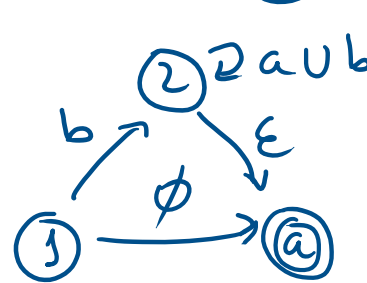
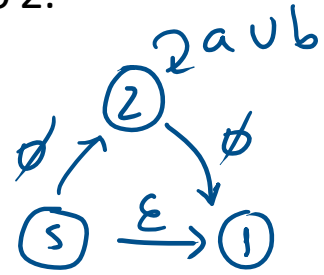
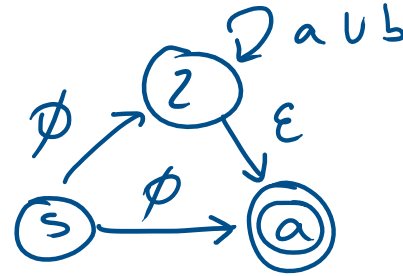
1ª) gerar todos os pares de estados passando pelo estado 2:



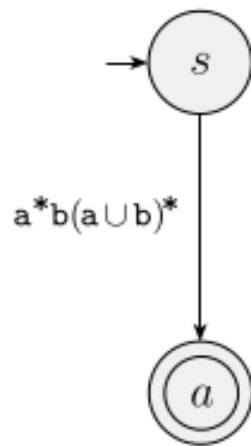
(a)



(b)



(c)

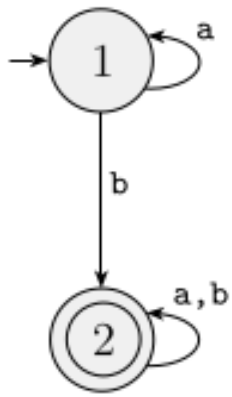


(d)

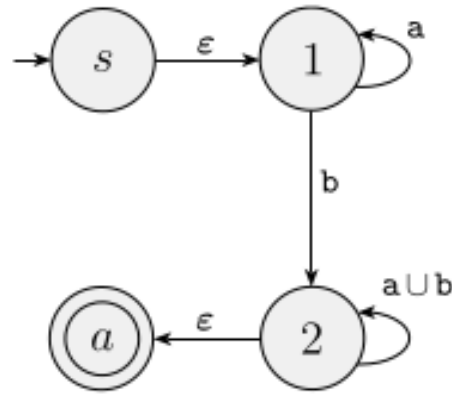
Algoritmo de conversão

Para remover o estado 2.

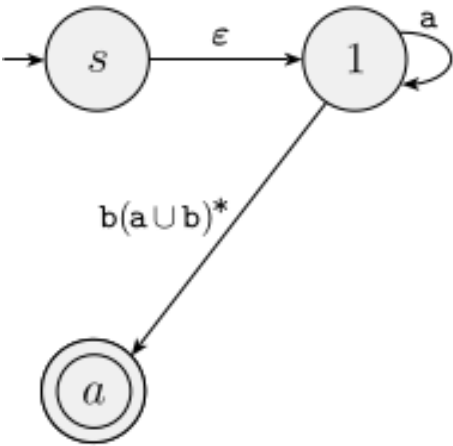
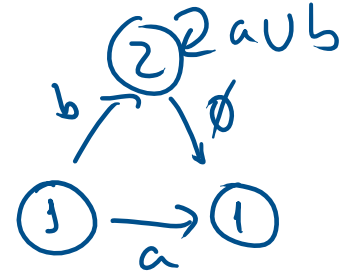
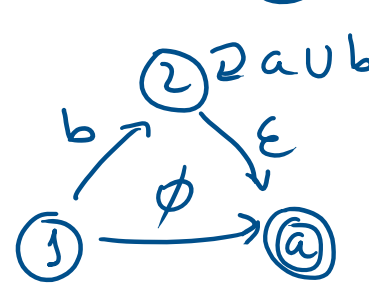
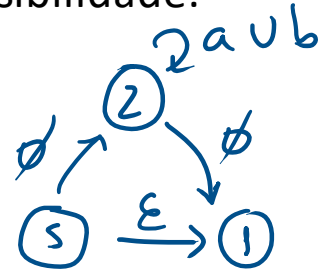
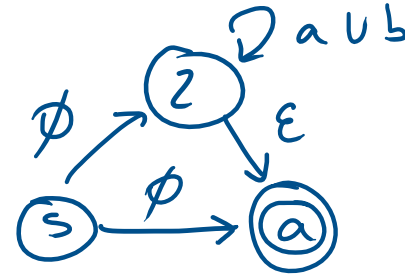
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



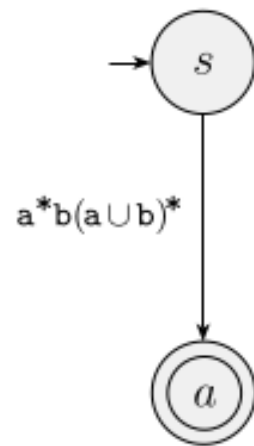
(a)



(b)



(c)

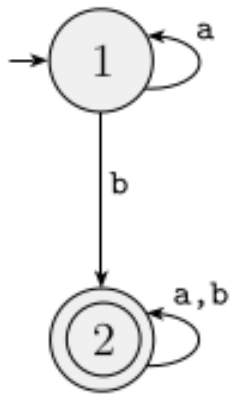


(d)

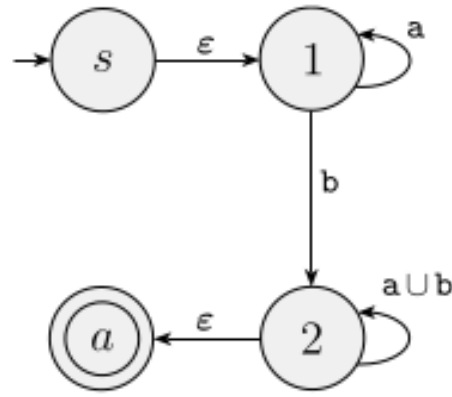
Algoritmo de conversão

Para remover o estado 2.

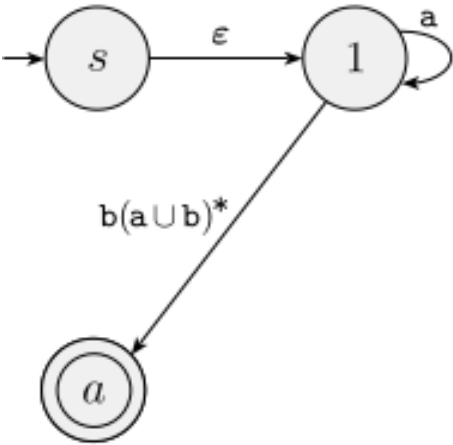
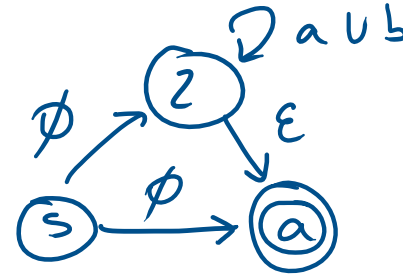
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



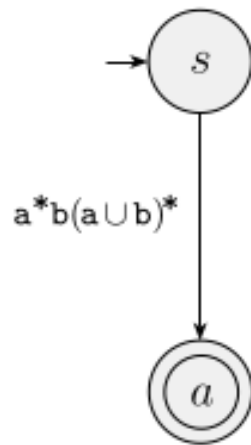
(a)



(b)



(c)



(d)

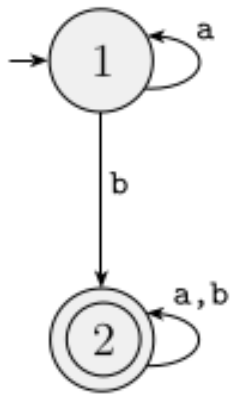
Handwritten calculation in red ink:

$$\hookrightarrow \textcircled{s} \quad \textcircled{a} \quad \emptyset (a \cup b)^* \epsilon \cup \emptyset = \emptyset$$

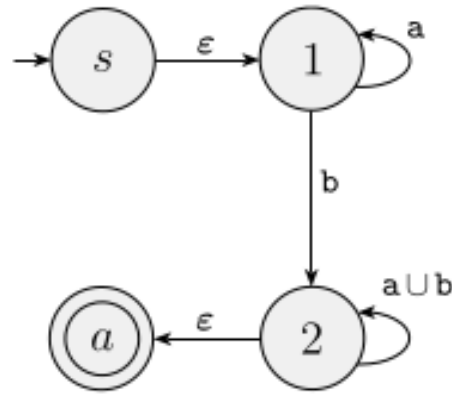
Algoritmo de conversão

Para remover o estado 2.

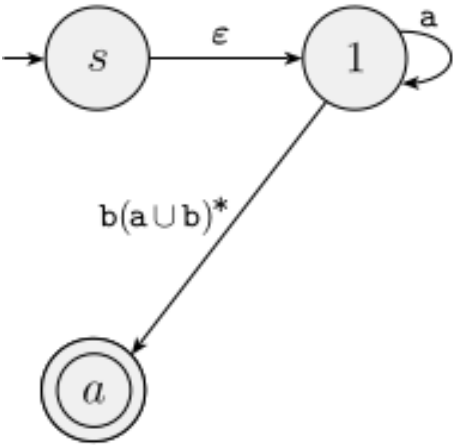
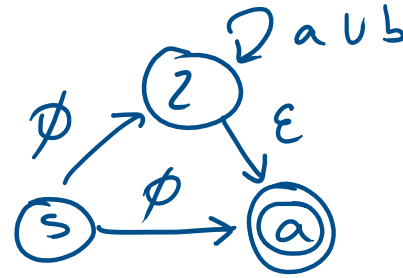
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



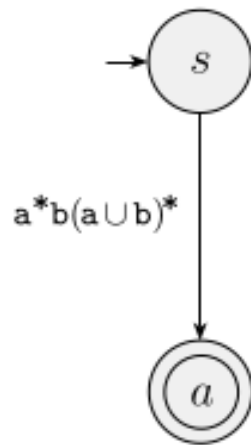
(a)



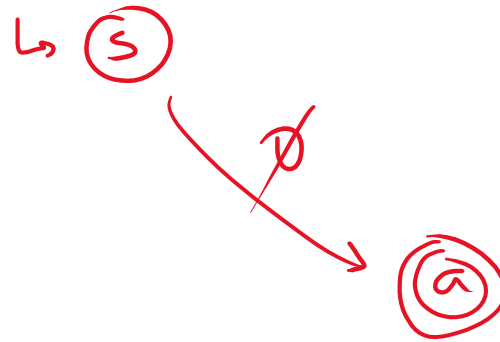
(b)



(c)



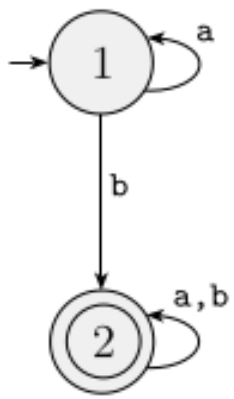
(d)



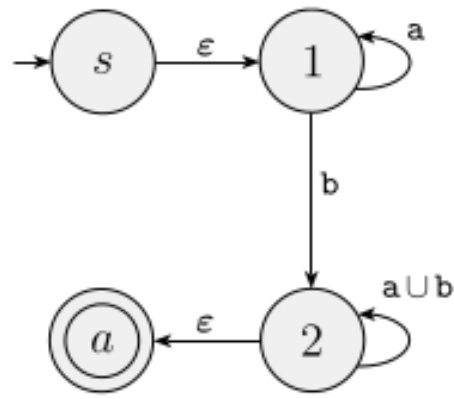
Algoritmo de conversão

Para remover o estado 2.

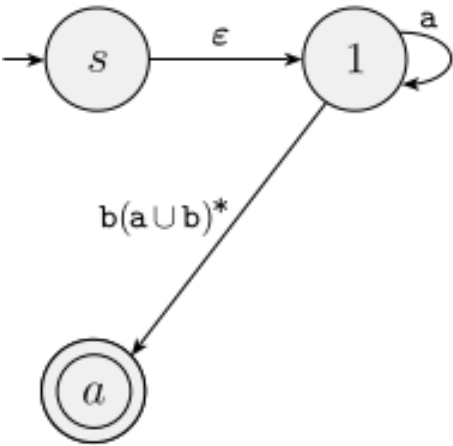
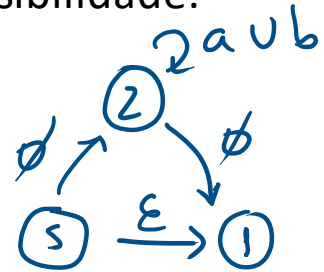
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



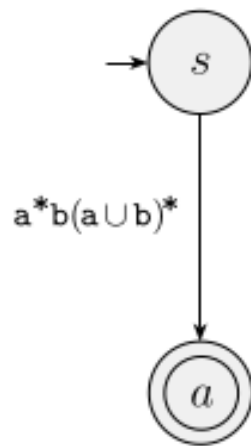
(a)



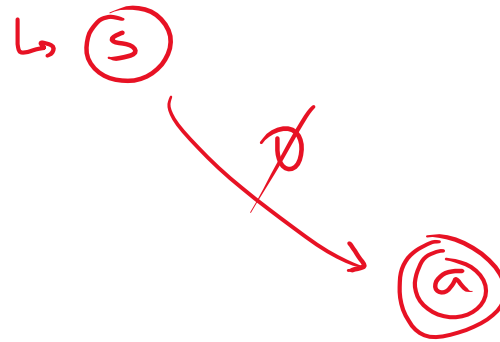
(b)



(c)



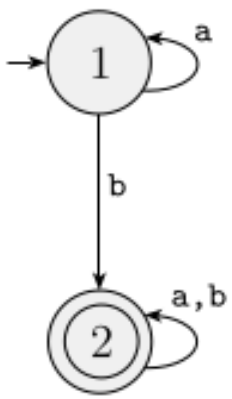
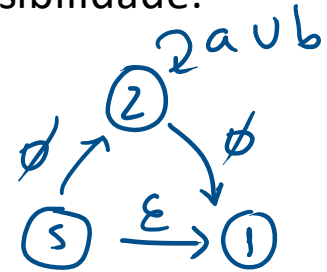
(d)



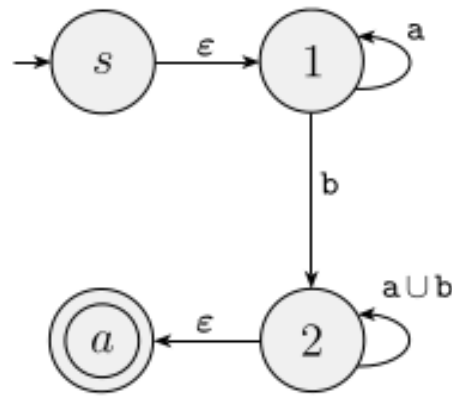
Algoritmo de conversão

Para remover o estado 2.

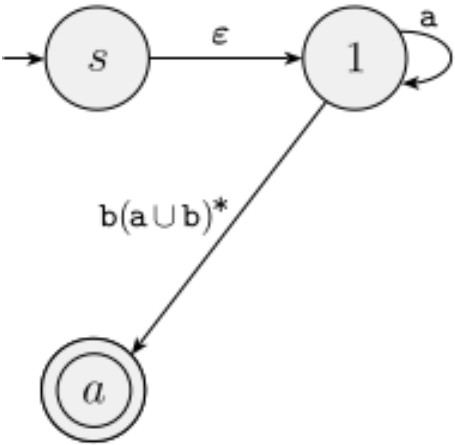
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



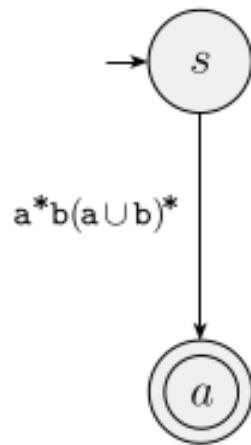
(a)



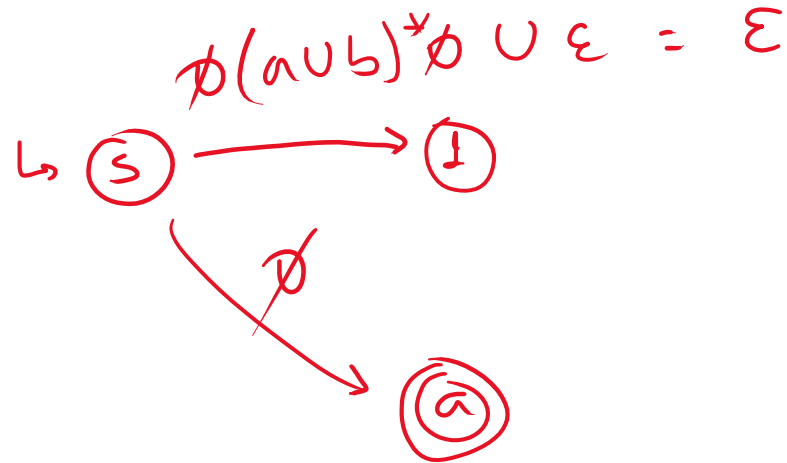
(b)



(c)



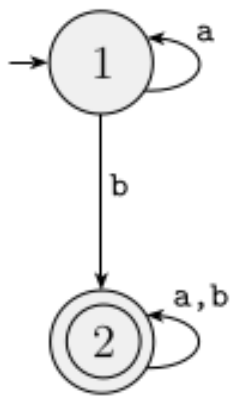
(d)



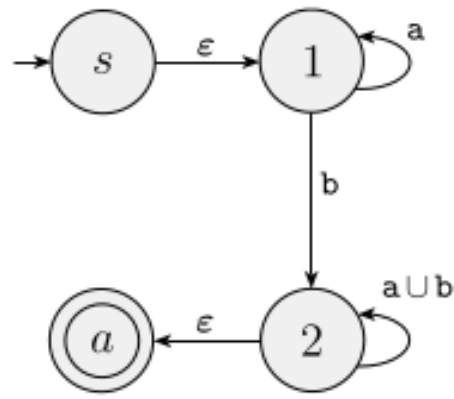
Algoritmo de conversão

Para remover o estado 2.

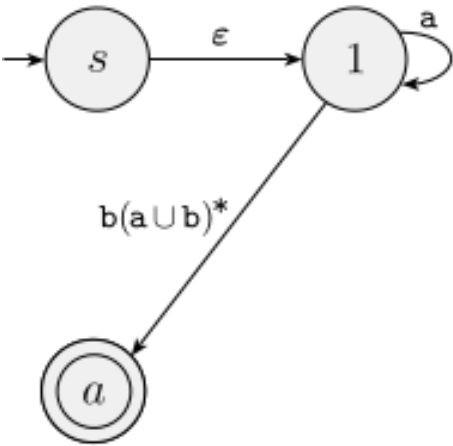
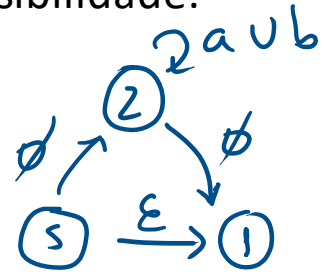
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



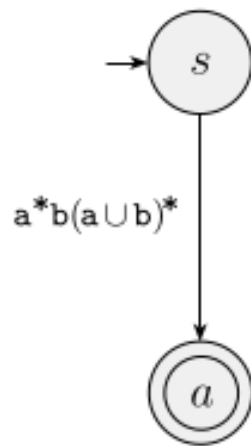
(a)



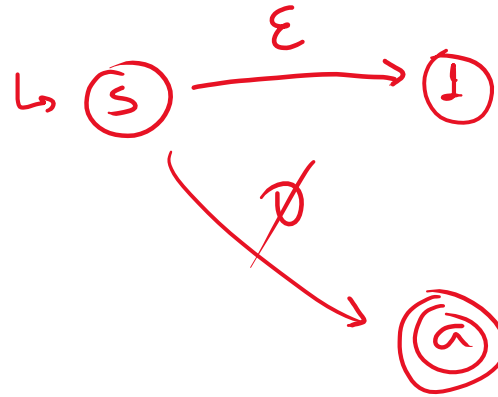
(b)



(c)



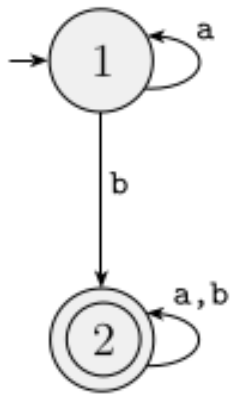
(d)



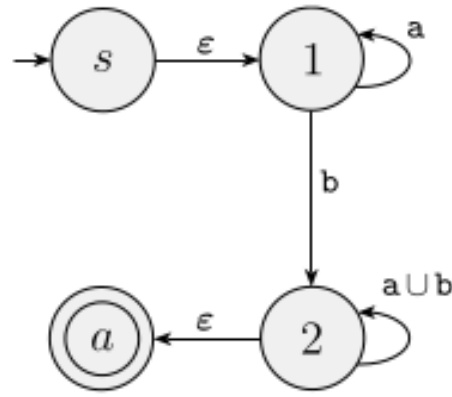
Algoritmo de conversão

Para remover o estado 2.

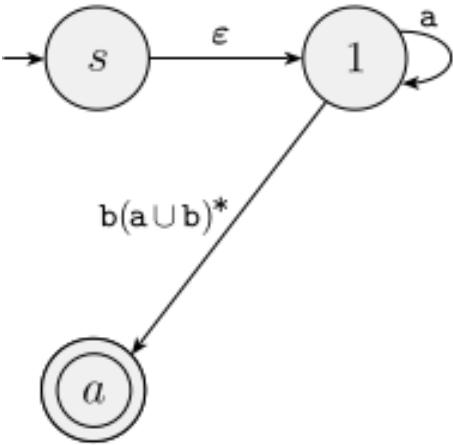
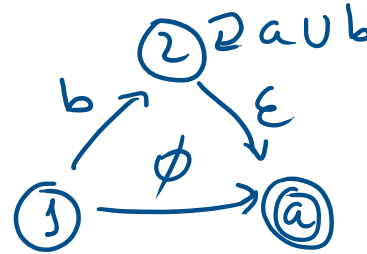
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



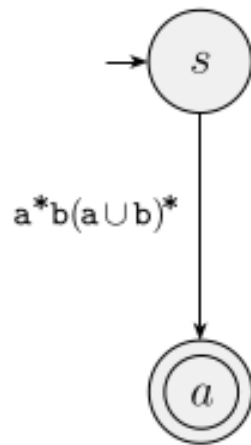
(a)



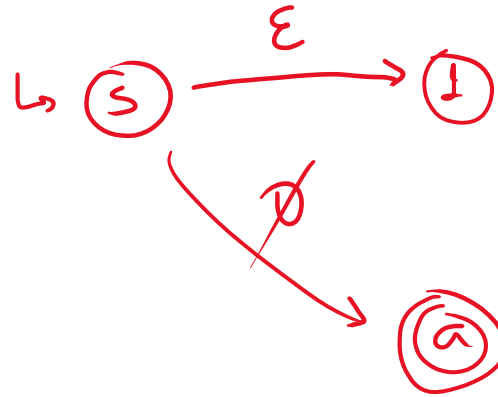
(b)



(c)



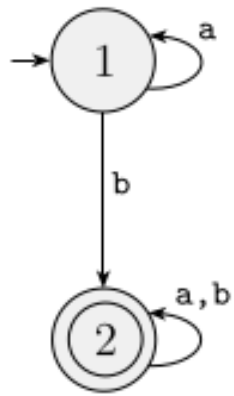
(d)



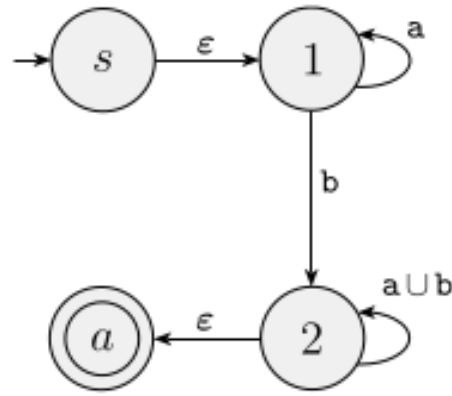
Algoritmo de conversão

Para remover o estado 2.

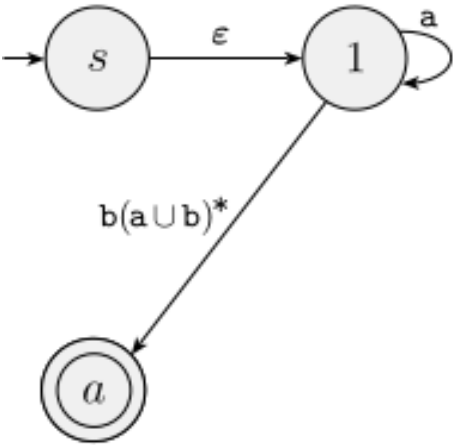
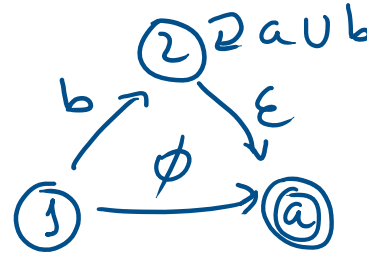
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



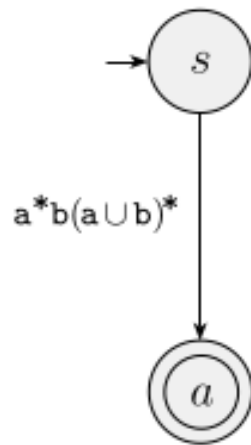
(a)



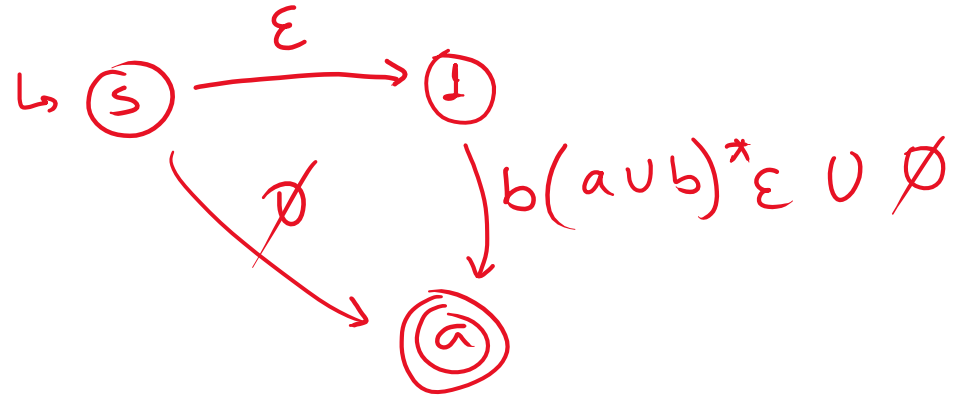
(b)



(c)



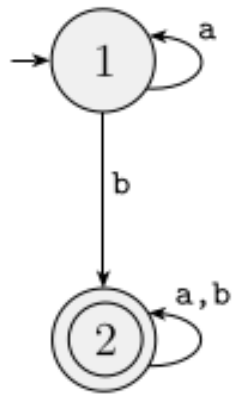
(d)



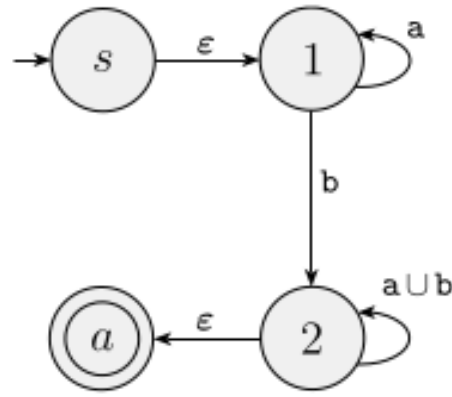
Algoritmo de conversão

Para remover o estado 2.

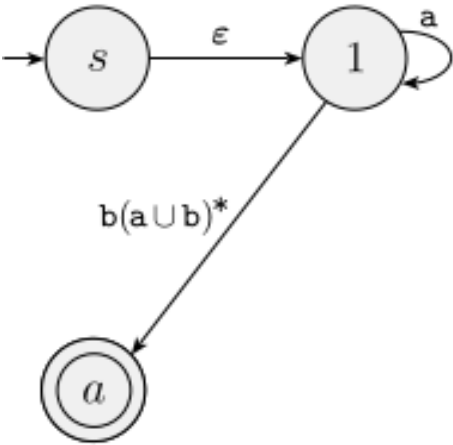
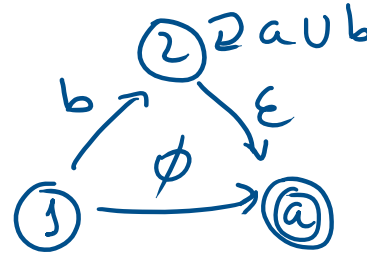
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



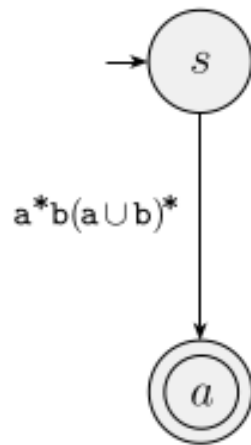
(a)



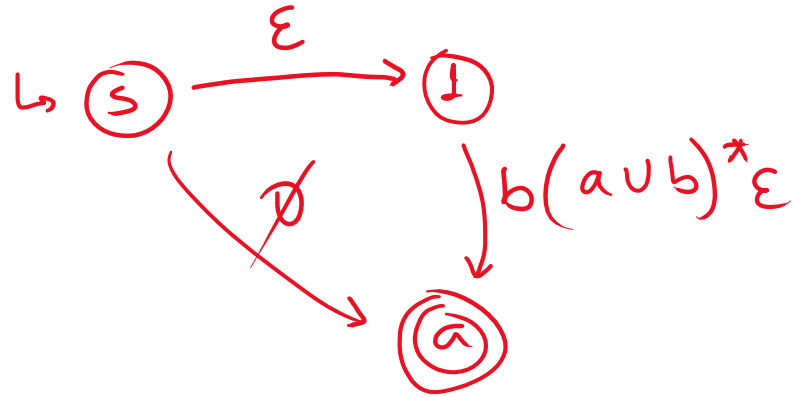
(b)



(c)



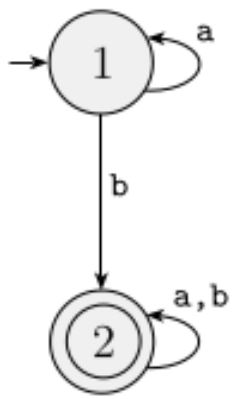
(d)



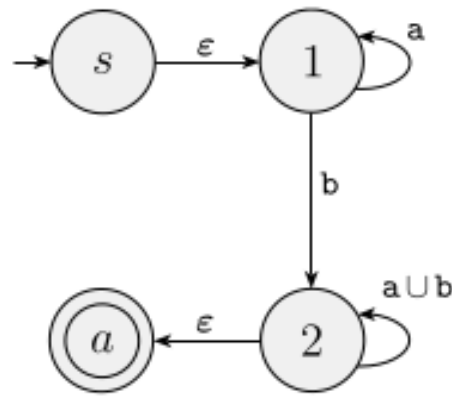
Algoritmo de conversão

Para remover o estado 2.

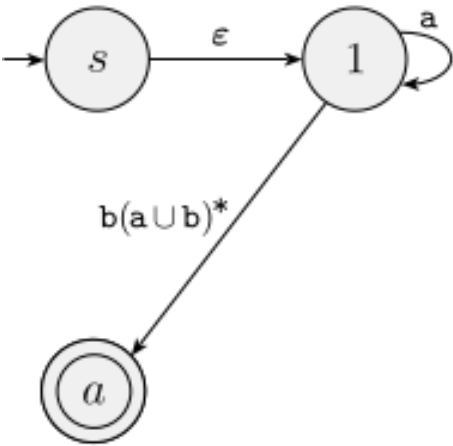
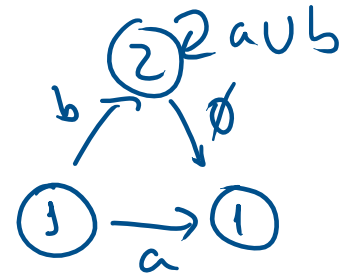
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



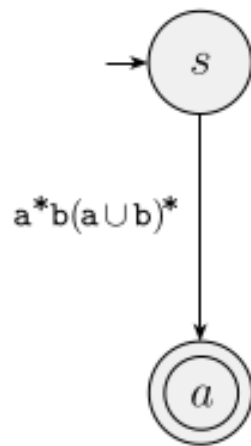
(a)



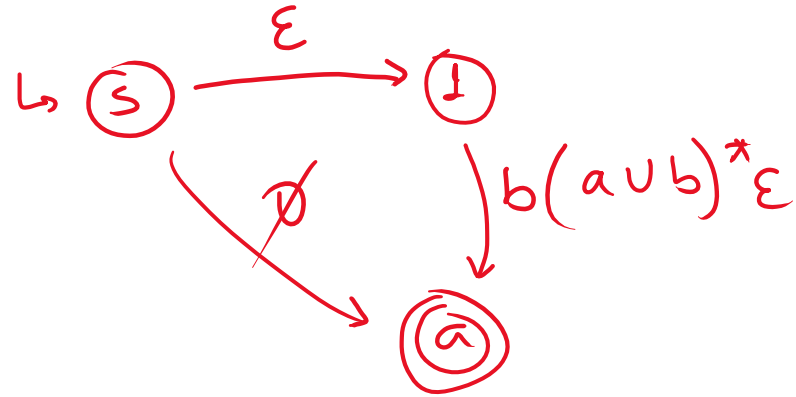
(b)



(c)



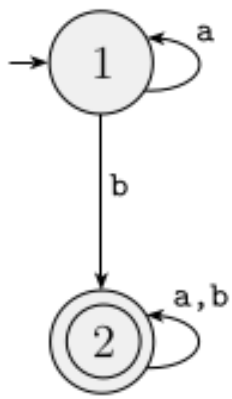
(d)



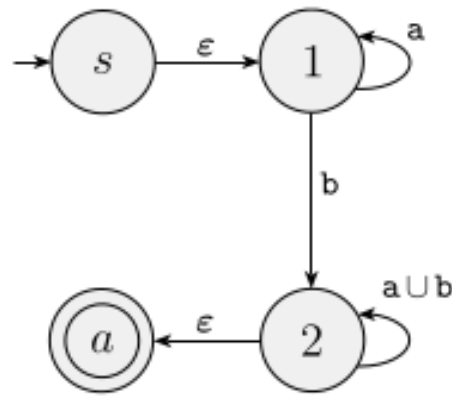
Algoritmo de conversão

Para remover o estado 2.

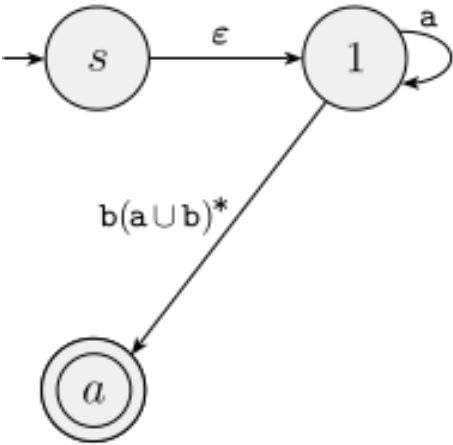
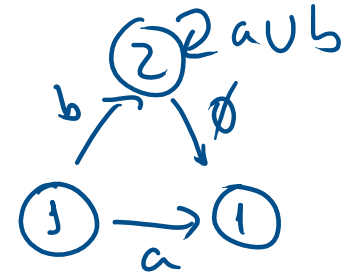
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



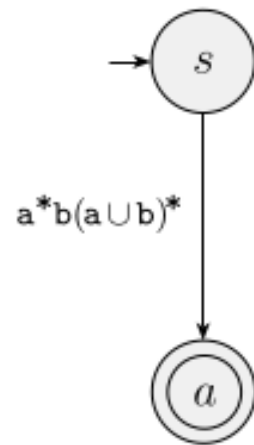
(a)



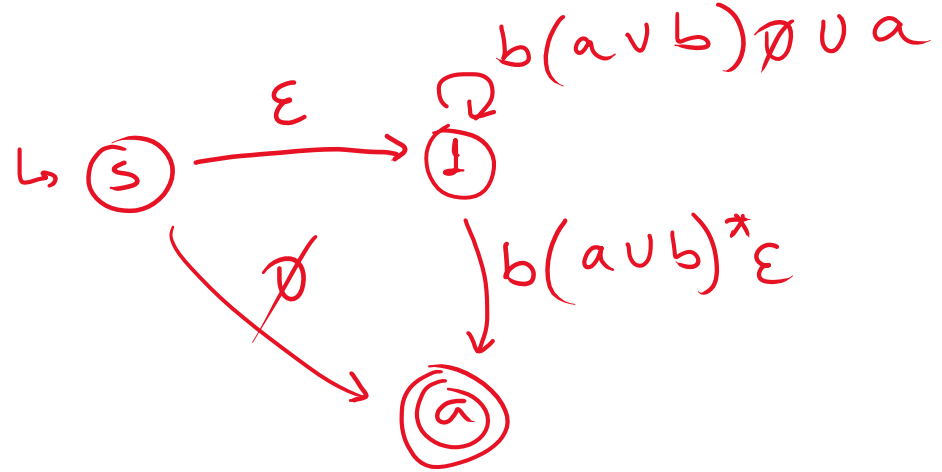
(b)



(c)



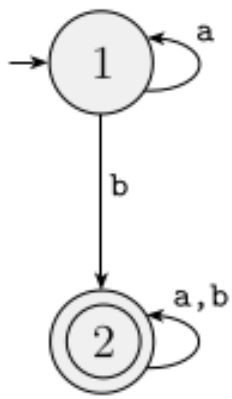
(d)



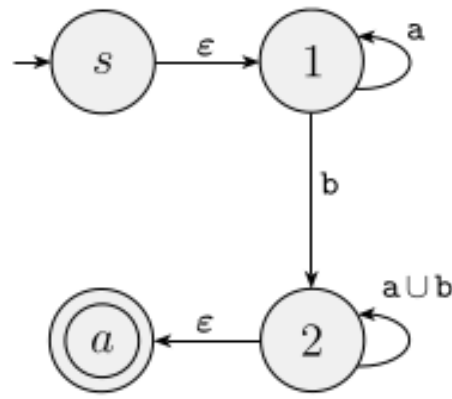
Algoritmo de conversão

Para remover o estado 2.

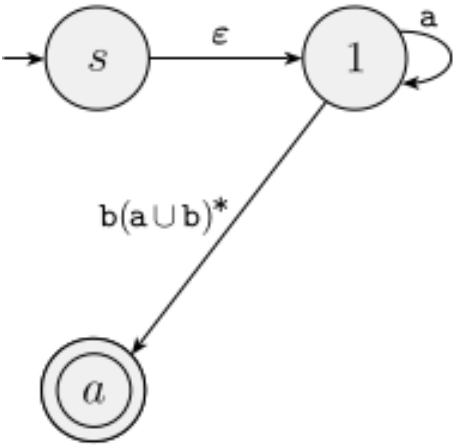
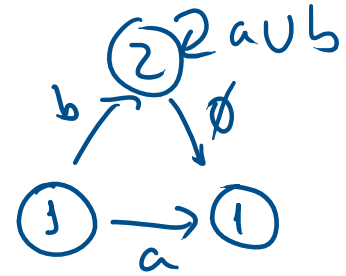
2º) passo 3 do algoritmo, que remove o estado 2 de cada possibilidade:



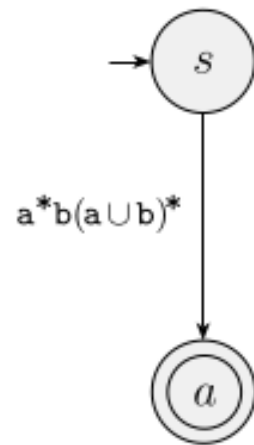
(a)



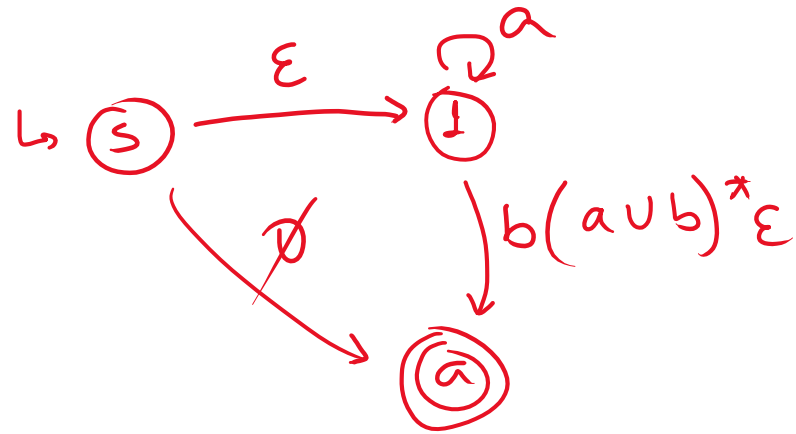
(b)



(c)



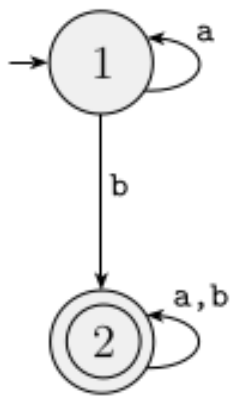
(d)



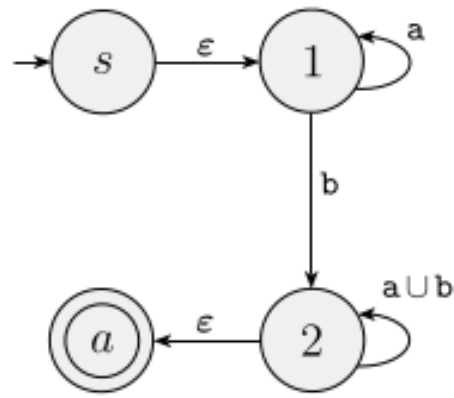
Algoritmo de conversão

Para remover o estado 1.

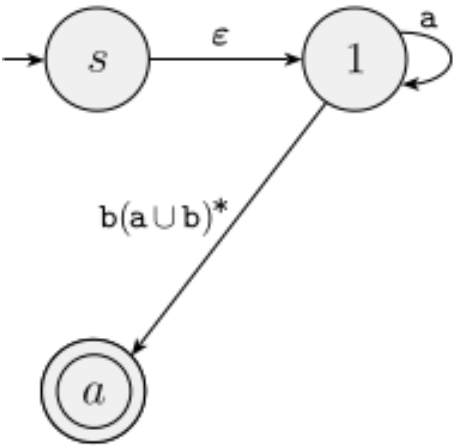
1º) listar todas as possibilidades de caminhos passando pelo 1.



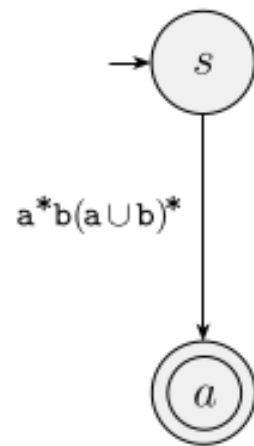
(a)



(b)



(c)

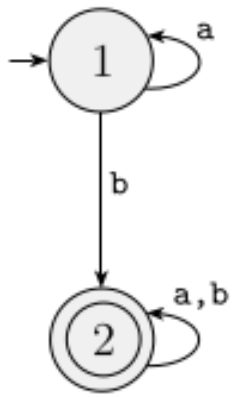
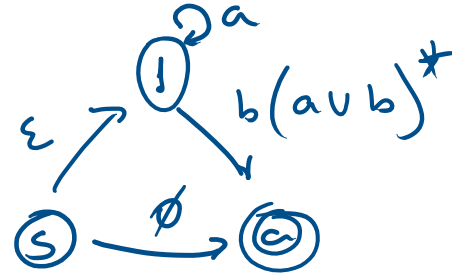


(d)

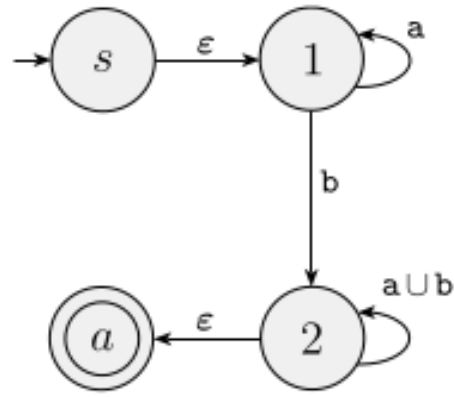
Algoritmo de conversão

Para remover o estado 1.

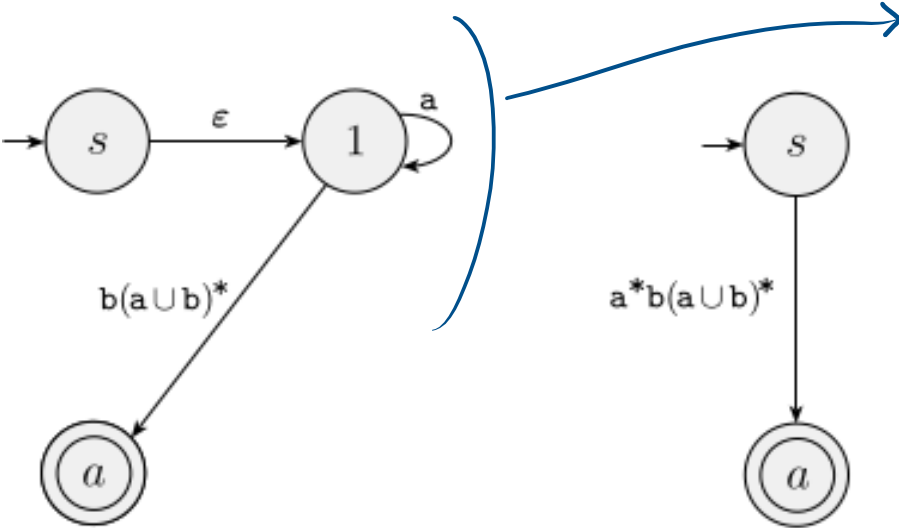
1º) listar todas as possibilidades de caminhos passando pelo 1.



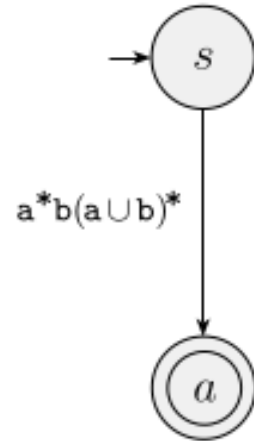
(a)



(b)



(c)

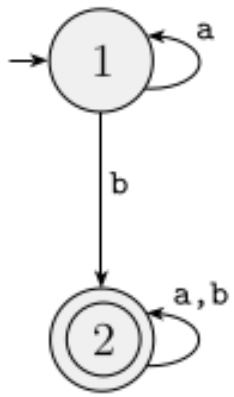
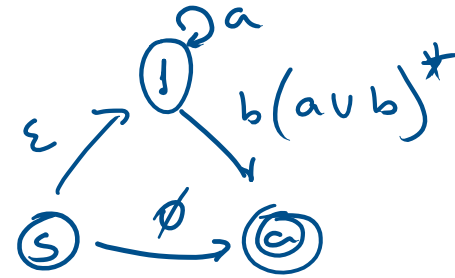


(d)

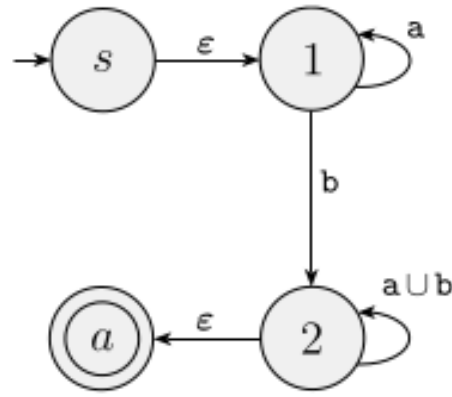
Algoritmo de conversão

Para remover o estado 1.

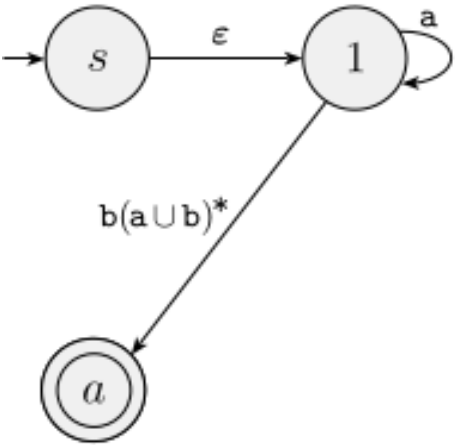
1º) listar todas as possibilidades de caminhos passando pelo 1.



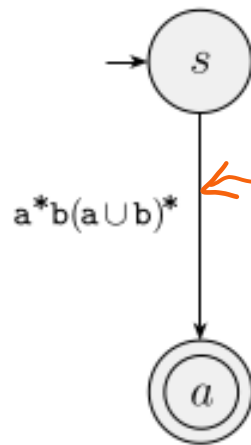
(a)



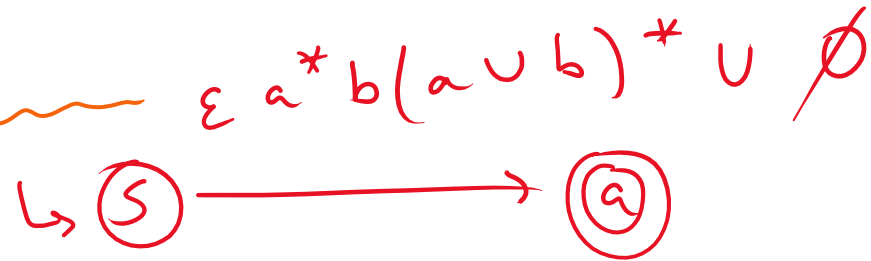
(b)



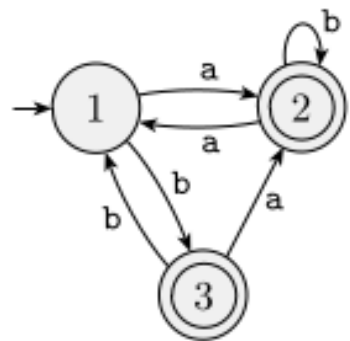
(c)



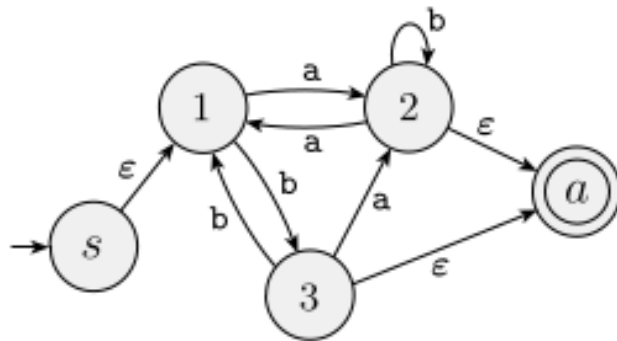
(d)



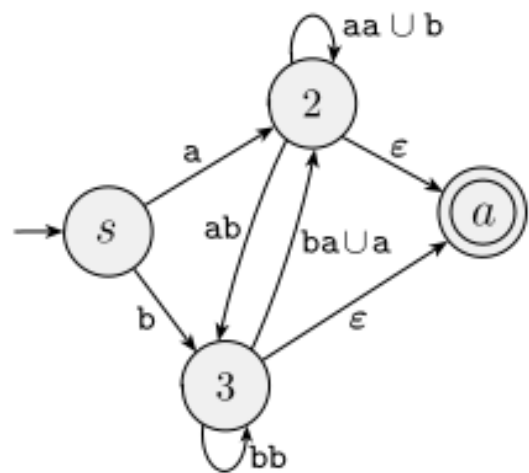
Exemplo 1.68



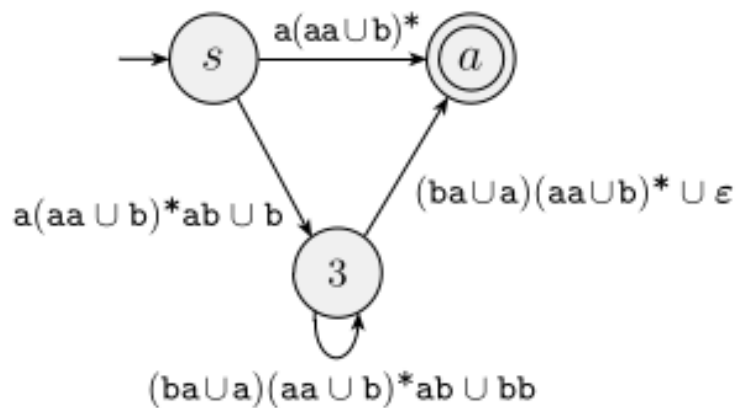
(a)



(b)



(c)



(d)